

Node.js 교과서



Node.js 교과서 개정2판

길벗

10장

10.1 API 서버 이해하기

10.2 프로젝트 구조 갖추기

10.3 JWT 토큰으로 인증하기

10.4 호출 서버 만들기

10.5 SNS API 서버 만들기

10.6 사용량 제한 구현하기

10.7 CORS 이해하기

10.1 API 서버 이해하기

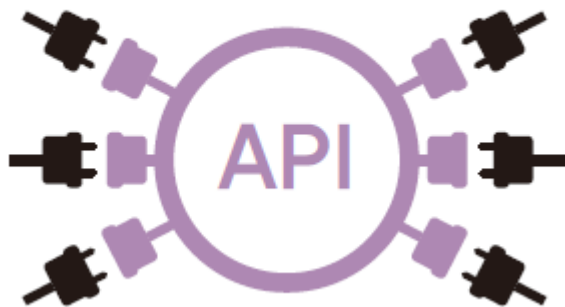
1. NodeBird SNS 서비스

» 9 장 NodeBird 서비스의 API 서버를 만듭

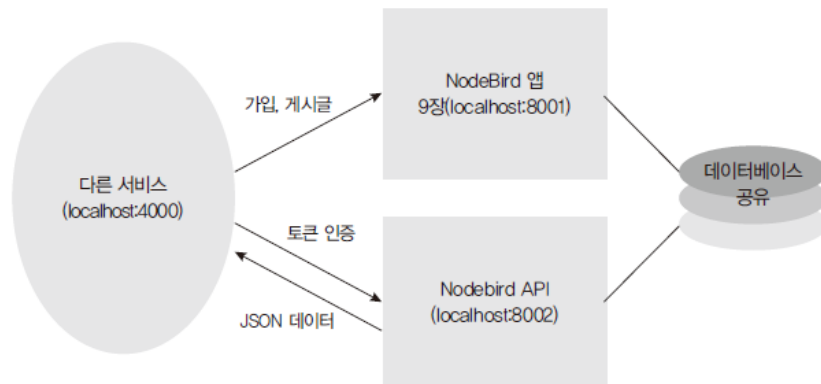
» API: Application Programming Interface

- 다른 애플리케이션에서 현재 프로그램의 기능을 사용할 수 있게 함
- 웹 API: 다른 웹 서비스의 기능을 사용하거나 자원을 가져올 수 있게 함
- 다른 사람에게 정보를 제공하고 싶은 부분만 API를 열고, 제공하고 싶지 않은 부분은 API를 만들지 않으면 됨
- API에 제한을 걸어 일정 횟수 내에서만 가져가게 할 수도 있음
- NodeBird에서는 인증된 사용자에게만 정보 제공

▼ 그림 10-1 API 개념도



▼ 그림 10-2 전체 클라이언트 서버 관계도



10.2 프로젝트 구조 갖추기



1. NodeBird API 폴더 세팅

» nodebird-api 폴더를 만들고 package.json 생성

- 생성 후 npm i로 패키지 설치
- NodeBird에서 config, models, passport 모두 복사해서 nodebird-api에 붙여넣기
- routes 폴더에서는 auth.js와 middlewares 재사용
- .env 파일 복사
- nodebird-api/views/error.html or.html 파일 생성

```
<h1>{{message}}</h1>
<h2>{{error.status}}</h2>
<pre>{{error.stack}}</pre>
```

nodebird-api/package.json

```
{
  "name": "nodebird-api",
  "version": "0.0.1",
  "description": "NodeBird API 서버",
  "main": "app.js",
  "scripts": {
    "start": "nodemon app"
  },
  "author": "Zero Cho",
  "license": "ISC",
  "dependencies": {
    "bcrypt": "^5.0.0",
    "cookie-parser": "^1.4.5",
    "dotenv": "^8.2.0",
    "express": "^4.17.1",
    "express-session": "^1.17.1",
    "morgan": "^1.10.0",
    "mysql2": "^2.1.0",
    "nunjucks": "^3.2.1",
    "passport": "^0.4.1",
    "passport-kakao": "1.0.0",
    "passport-local": "^1.0.0",
    "sequelize": "^6.2.3",
    "uuid": "^8.2.0"
  },
  "devDependencies": {
    "nodemon": "^2.0.3"
  }
}
```



2. app.js 생성하기

» 소스 코드는 <https://github.com/zerocho/nodejs-book>

» 8002번 포트 사용

- 8001번을 사용하는 NodeBird 서비스와 8003을 사용할 nodecat 서비스와 함께 사용할 수 있음
- 콘솔을 여러 개 실행해 각각의 서비스를 돌리면 됨

» nodebird-api/views/login.html 화면 생성

- NodeBird 서비스의 계정으로 로그인하면 됨(카카오톡 로그인 안 됨)



3. 도메인 모델 생성하기

» models/domain.js 작성

- API를 사용할 도메인(또는 호스트)을 저장하는 모델
- ENUM type으로 free나 premium만 쓸 수 있게 제한
- clientSecret은 uuid 타입으로

nodebird-api/models/domain.js

```
const Sequelize = require('sequelize');

module.exports = class Domain extends Sequelize.Model {
  static init(sequelize) {
    return super.init({
      host: {
        type: Sequelize.STRING(80),
        allowNull: false,
      },
      type: {
        type: Sequelize.ENUM('free', 'premium'),
        allowNull: false,
      },
      clientSecret: {
        type: Sequelize.UUID,
        allowNull: false,
      },
    }, {
      sequelize,
      timestamps: true,
      paranoid: true,
      modelName: 'Domain',
      tableName: 'domains',
    });
  }

  static associate(db) {
    db.Domain.belongsTo(db.User);
  }
};
```




4. 도메인 등록 라우터

» routes/index에서 도메인 등록 라우터 생성

- uuid 패키지로 사용자가 등록한 도메인에 고유한 비밀번호 부여
- uuid는 충돌(고유하지 않은 상황) 위험이 있지만 매우 희박
- 비밀번호가 일치하는 요청만 API 응답

nodebird-api/routes/index.js

```
const express = require('express');
const { v4: uuidv4 } = require('uuid');
const { User, Domain } = require('../models');
const { isLoggedIn } = require('./middlewares');

const router = express.Router();

router.get('/', async (req, res, next) => {
  try {
    const user = await User.findOne({
      where: { id: req.user.id || req.user.id || null },
      include: { model: Domain },
    });
    res.render('login', {
      user,
      domains: user && user.Domains,
    });
  } catch (err) {
    console.error(err);
    next(err);
  }
});

router.post('/domain', isLoggedIn, async (req, res, next) => {
  try {
    await Domain.create({
      UserId: req.user.id,
      host: req.body.host,
      type: req.body.type,
      clientSecret: uuidv4(),
    });
    res.redirect('/');
  } catch (err) {
    console.error(err);
    next(err);
  }
});

module.exports = router;
```



5. 도메인 등록하고 비밀번호 발급받기

» 라우터 작성 후 localhost:8002 접속

- 도메인이 다른 프론트엔드에서 요청을 보내면 CORS 에러(10.7) 발생
- 로그인 후 localhost:8003(nodecat 서버) 등록

▼ 그림 10-3 http://localhost:8002 접속 화면

NodeBird 계정으로 로그인하세요.

이메일

비밀번호

회원가입은 localhost:8001에서 하세요.

▼ 그림 10-4 도메인 등록 화면

안녕하세요! zero님

도메인 등록

무료 ☒ 프리미엄 ☐

도메인(ex) zerocho.com

▼ 그림 10-5 도메인 등록 후 화면

안녕하세요! 제로초님

도메인 등록

무료 ☒ 프리미엄 ☐

도메인

도메인 주소 타입 클라이언트 비밀번호

localhost:4000 free 7d67444e-fd01-4f9b-8680-f72464d02a57

10.3 JWT 토큰으로 인증하기



1. 인증을 위한 JWT

» NodeBird가 아닌 다른 클라이언트가 데이터를 가져가게 하려면 인증 과정이 필요함

» JWT(JSON Web Token)을 사용함

- 헤더.페이로드.시그니처로 구성됨
- **헤더**: 토큰 종류와 해시 알고리즘 정보가 들어있음
- **페이로드**: 토큰의 내용물이 인코딩된 부분
- **시그니처**: 일련의 문자열로, 시그니처를 통해 토큰이 변조되었는지 여부 확인
- 시그니처는 JWT 비밀키를 마트십지크 비밀키가 1 초이면 63 이자 가나

JWT 토큰 예시

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjEiLCJuYW11Ijo7KGw7ZiE7JiBIiwibWVzc2FnZSI6IiUuwl0uztCJ9.7IIeFTuda7i66CoiabRSnoVylj0V9UqAetMwpidpXbw
```

2. JWT 사용 시 주의점

» JWT에 민감한 내용을 넣으면 안 됨

- 페이로드 내용 볼 수 있음

The image shows the JWT.io token decoder interface. At the top, the 'ALGORITHM' is set to 'HS256'. The 'Encoded' section on the left contains a long base64-encoded string. The 'Decoded' section on the right shows the token's structure:

- HEADER: ALGORITHM & TOKEN TYPE:**

```
{  "alg": "HS256",  "typ": "JWT"}
```
- PAYLOAD: DATA:**

```
{  "id": "1",  "name": "조현영",  "message": "나보"}
```
- VERIFY SIGNATURE:**

```
HS256(base64urlEncode(header) + "." + base64urlEncode(payload), secret)
```

At the bottom left, it says 'Signature Verified' with a checkmark icon. At the bottom right, there is a blue button labeled 'SHARE JWT'.

- 그럼에도 사용하는 이유는 토큰 변조가 불가능하고, 내용물이 들어있기 때문
- 내용물이 들어있으므로 데이터베이스 조회를 하지 않을 수 있음(데이터베이스 조회는 비용이 큰 작업)
- 노출되어도 좋은 정보만 넣어야 함
- 용량이 커서 요청 시 데이터 양이 증가한다는 단점이 있음



3. 노드에서 JWT 사용하기

» JWT 모듈 설치

- npm i jsonwebtoken
- JWT 비밀키 .env에 저장

nodebird-api/.env

COOKIE_SECRET=nodebirdsecret

KAKAO_ID=5d4daf57becfd72fd9c919882552c4a6

JWT_SECRET=jwtSecret

nodebird-api/routes/middlewares.js

```
const jwt = require('jsonwebtoken');
...
exports.verifyToken = (req, res, next) => {
  try {
    req.decoded = jwt.verify(req.headers.authorization, process.env.JWT_SECRET);
    return next();
  } catch (error) {
    if (error.name === 'TokenExpiredError') { // 유효 기간 초과
      return res.status(419).json({
        code: 419,
        message: '토큰이 만료되었습니다',
      });
    }
    return res.status(401).json({
      code: 401,
      message: '유효하지 않은 토큰입니다',
    });
  }
};
```

- JWT 토큰을 검사하는 verifyToken 미들웨어 작성
- jwt.verify 메서드로 검사 가능(두 번째 인수가 JWT 비밀키)
- JWT 토큰은 req.headers.authorization에 들어 있음
- 만료된 JWT 토큰인 경우 419 에러 발생
- 유효하지 않은 토큰인 경우 401에러 발생
- req.decoded에 페이로드를 넣어 다음 미들웨어에서 쓸 수 있게 함

4. JWT 토큰 발급 라우터 만들기

» routes/v1.js 작성

- 버전 1이라는 뜻의 v1.js
- 한 번 버전이 정해진 후에는 라우터를 함부로 수정하면 안 됨
- 다른 사람이 기존 API를 쓰고 있기 때문(그 사람에게 영향이 감)
- 수정 사항이 생기면 버전을 올려야 함
- POST /token에서 JWT 토큰 발급
- 먼저 도메인 검사 후 등록된 도메인이면 jwt.sign 메서드로 JWT 토큰 발급
- 첫 번째 인수로 페이로드를 넣고, 두 번째 인수는 JWT 비밀키, 세 번째 인수로 토큰 옵션(expiresIn은 만료 시간, issuer은 발급자)
- expiresIn은 1m(1분), 60 * 1000같은 밀리초 단위도 가능
- GET /test 라우터에서 토큰 인증 테스트 가능

```
const token = jwt.sign({  
  id: domain.user.id,  
  nick: domain.user.nick,  
}, process.env.JWT_SECRET, {  
  expiresIn: '1m', // 유효 기간  
  issuer: 'nodebird', // 발급자  
});
```



5. app.js에 라우터 연결

» app.js에 v1 라우터 연결

nodebird-api/app.js

```
...  
const dotenv = require('dotenv');  
const v1 = require('./routes/v1');  
const authRouter = require('./routes/auth');  
  
...  
app.use(passport.session());  
  
app.use('/v1', v1);  
app.use('/auth', authRouter);
```




6. JWT 토큰으로 로그인하기

» 세션 쿠키 발급 대신 JWT 토큰을 쿠키로 발급하면 됨

- Authenticate 메서드의 두 번째 인수로 옵션을 주면 세션 사용하지 않음

```
...
router.post('/login', isNotLoggedIn, (req, res, next) => {
  passport.authenticate('local', { session: false }, (authError, user, info) => {
    if (authError) {

```

» 클라이언트에서 JWT를 사용하고 싶다면

- process.env.JWT_SECRET은 클라이언트에서 노출되면 안 됨
- RSA같은 양방향 비대칭 암호화 알고리즘을 사용해야 함
- JWT는 PEM 키를 사용해서 양방향 암호화를 하는 것을 지원함

10.4 호출 서버 만들기



1. API 호출용 서버 만들기

» nodecat 폴더 만들고 package.json 파일을 만들

nodecat/package.json

```
{
  "name": "nodecat",
  "version": "0.0.1",
  "description": "노드버드 2차 서비스",
  "main": "app.js",
  "scripts": {
    "start": "nodemon app"
  },
  "author": "Zero Cho",
  "license": "ISC",
  "dependencies": {
    "axios": "^0.19.2",
    "cookie-parser": "^1.4.5",
    "dotenv": "^8.2.0",
    "express": "^4.17.1",
    "express-session": "^1.17.1",
    "morgan": "^1.10.0",
    "nunjucks": "^3.2.1"
  },
  "devDependencies": {
    "nodemon": "^2.0.3"
  }
}
```



2. 간단한 폴더 구조 갖추기

» app.js 파일 생성

- 소스 코드는 <https://github.com/zerocho/nodejs-book/ch10/10.4/nodecat>
- views/error.html는 nodebird-api로부터 복사
- 아까 발급받은 비밀키를 .env에 입력

```
nodecat/.env
```

```
COOKIE_SECRET=nodecat
```

```
CLIENT_SECRET=7d67444e-fd01-4f9b-8680-f72464d02a57
```



3. 토큰 테스트용 라우터 만들기

» routes/index.js 생성

- GET /test에 접근 시 세션 검사
- 세션에 토큰이 저장되어 있지 않으면
POST <http://localhost:8002/v1/token>
라우터로부터 토큰 발급
- 이 때 HTTP 요청 본문에 클라이언트
비밀키 동봉
- 발급에 성공했다면 발급받은
토큰으로 다시 GET
<https://localhost:8002/v1/test> 라우터
접근해서 토큰 테스트

nodecat/routes/index.js

```
const express = require('express');
const axios = require('axios');

const router = express.Router();

router.get('/test', async (req, res, next) => { // 토큰 테스트 라우터
  try {
    if (!req.session.jwt) { // 세션에 토큰이 없으면 토큰 발급 시도
      const tokenResult = await axios.post('http://localhost:8002/v1/token', {
        clientSecret: process.env.CLIENT_SECRET,
      });
      if (tokenResult.data && tokenResult.data.code === 200) { // 토큰 발급 성공
        req.session.jwt = tokenResult.data.token; // 세션에 토큰 저장
      } else { // 토큰 발급 실패
        return res.json(tokenResult.data); // 발급 실패 사유 응답
      }
    }
    // 발급받은 토큰 테스트
    const result = await axios.get('http://localhost:8002/v1/test', {
      headers: { authorization: req.session.jwt },
    });
    return res.json(result.data);
  } catch (error) {
    console.error(error);
    if (error.response.status === 419) { // 토큰 만료 시
      return res.json(error.response.data);
    }
    return next(error);
  }
});

module.exports = router;
```

4. 실제 요청 보내기

» npm start로 서버 시작

- <http://localhost:4000/test>로 접속

NodeCat과 NodeBird API 서버에서 모두 실행

```
$ npm start
// 각각 다른 포트에서 대기 중
4000 번 포트에서 대기 중
8002 번 포트에서 대기 중
```

▼ 그림 10-7 토큰 테스트 화면

```
{"id":1,"nick":"제로초","iat":1577251700,"exp":1577251760,"iss":"nodebird"}
```

- 1분을 기다린 후 다시 접속하면 토큰이 만료되었다는 메시지 뜸

▼ 그림 10-8 토큰 만료 화면

```
{"code":419,"message":"토큰이 만료되었습니다"}
```

10.5 SNS API 서버 만들기



1. NodeBird 데이터 제공하기

» nodebird-api의 라우터 작성

- GET /posts/my, GET /posts/hashtag/:title

nodebird-api/routes/v1.js

```
const express = require('express');
const jwt = require('jsonwebtoken');

const { verifyToken } = require('./middlewares');
const { Domain, User, Post, Hashtag } = require('../models');

const router = express.Router();
...
router.get('/posts/my', verifyToken, (req, res) => {
  Post.findAll({ where: { userId: req.decoded.id } })
    .then((posts) => {
      console.log(posts);
      res.json({
        code: 200,
        payload: posts,
      });
    })
    .catch((error) => {
      console.error(error);
      return res.status(500).json({
        code: 500,
        message: '서버 에러',
      });
    });
});
```

```
router.get('/posts/hashtag/:title', verifyToken, async (req, res) => {
  try {
    const hashtag = await Hashtag.findOne({ where: { title: req.params.title } });
    if (!hashtag) {
      return res.status(404).json({
        code: 404,
        message: '검색 결과가 없습니다',
      });
    }
    const posts = await hashtag.getPosts();
    return res.json({
      code: 200,
      payload: posts,
    });
  } catch (error) {
    console.error(error);
    return res.status(500).json({
      code: 500,
      message: '서버 에러',
    });
  }
});

module.exports = router;
```




2. NodeBird 데이터 가져오기

» nodecat의 라우터 작성

- 토큰을 발급받고 요청을 보내는 부분을 request 함수로 만들어 둬
- 요청은 axios로 보내고 세션 토큰 검사, 재발급까지 같이 수행

nodecat/routes/index.js

```
const express = require('express');
const axios = require('axios');

const router = express.Router();
const URL = 'http://localhost:8002/v1';
axios.defaults.headers.origin = 'http://localhost:4000'; // origin 헤더 추가
const request = async (req, api) => {
  try {
    if (!req.session.jwt) { // 세션에 토큰이 없으면
      const tokenResult = await axios.post(`${URL}/token`, {
        clientSecret: process.env.CLIENT_SECRET,
      });
      req.session.jwt = tokenResult.data.token; // 세션에 토큰 저장
    }
    return await axios.get(`${URL}${api}`, {
      headers: { authorization: req.session.jwt },
    }); // API 요청
  } catch (error) {
    if (error.response.status === 419) { // 토큰 만료 시 토큰 재발급 받기
      delete req.session.jwt;
      return request(req, api);
    } // 419 외의 다른 에러면
    return error.response;
  }
};
```

```
router.get('/mypost', async (req, res, next) => {
  try {
    const result = await request(req, '/posts/my');
    res.json(result.data);
  } catch (error) {
    console.error(error);
    next(error);
  }
});

router.get('/search/:hashtag', async (req, res, next) => {
  try {
    const result = await request(
      req, `/posts/hashtag/${encodeURIComponent(req.params.hashtag)}`,
    );
    res.json(result.data);
  } catch (error) {
    if (error.code) {
      console.error(error);
      next(error);
    }
  }
});

module.exports = router;
```



3. 실제 요청 보내기

» localhost:4000/mypost에 접속하면 게시글 받아옴(NodeBird 서비스에 게시글이 있어야 함)

▼ 그림 10-9 localhost:4000/mypost 접속 화면

```
{ "code": 200, "payload": [ { "id": 1, "content": "#노드 #익스프레스 배우기 전에  
#자바스크립트를 먼저 배우세요.", "img": "/img/1200px-  
Node.js_logo.svg1577071286491.png", "createdAt": "2019-12-  
23T03:21:27.000Z", "updatedAt": "2019-12-23T03:21:27.000Z", "UserId": 1 } ] }
```

» localhost:4000/search/노드 라우터에 접속하면 노드 해시태그 검색

▼ 그림 10-10 localhost:4000/search/노드 접속 화면

```
{ "code": 200, "payload": [ { "id": 1, "content": "#노드 #익스프레스 배우기 전에  
#자바스크립트를 먼저 배우세요.", "img": "/img/1200px-  
Node.js_logo.svg1577071286491.png", "createdAt": "2019-12-  
23T03:21:27.000Z", "updatedAt": "2019-12-  
23T03:21:27.000Z", "UserId": 1, "PostHashtag": { "createdAt": "2019-12-  
23T03:21:28.000Z", "updatedAt": "2019-12-  
23T03:21:28.000Z", "PostId": 1, "HashtagId": 1 } } ] }
```

nodebird-api 콘솔

POST /v1/posts/hashtag/노드 419 0.962 ms - 56

POST /v1/token 200 23.383 ms - 252

POST /v1/posts/hashtag/노드 200 8.288 ms - 395

10.6 사용량 제한 구현하기



1. 사용량 제한 구현하기

» DOS 공격 등을 대비해야 함

- 일정 시간동안 횟수 제한을 두어 무차별적인 요청을 막을 필요가 있음
- `npm i express-rate-limit`
- `apiLimiter` 미들웨어 추가
 - `windowMS`(기준 시간), `max`(허용 횟수), `delayMS`(호출 간격), `handler`(제한 초과 시 콜백

nodebird-api/routes/middlewares.js

```
const jwt = require('jsonwebtoken');
const RateLimit = require('express-rate-limit');
...
exports.apiLimiter = new RateLimit({
  windowMs: 60 * 1000, // 1분
  max: 1,
  delayMs: 0,
  handler(req, res) {
    res.status(this.statusCode).json({
      code: this.statusCode, // 기본값 429

```

```
    message: '1분에 한 번만 요청할 수 있습니다.',
  });
},
});

exports.deprecated = (req, res) => {
  res.status(410).json({
    code: 410,
    message: '새로운 버전이 나왔습니다. 새로운 버전을 사용하세요.',
  });
};
```

- `deprecated` 미들웨어는 사용하면 안 되는 라우터에 붙여서 사용 시 경고

2. 응답 코드 정리

» 응답 코드를 정리해서 어떤 에러가 발생했는지 알려주기

- 일관성이 있으면 됨

▼ 표 10-1 API 응답 목록

응답 코드	메시지
200	JSON 데이터입니다.
401	유효하지 않은 토큰입니다.
410	새로운 버전이 나왔습니다. 새로운 버전을 사용하세요.
419	토큰이 만료되었습니다.
429	1분에 한 번만 요청할 수 있습니다.
500~	기타 서버 에러



3. 새 라우터 버전 내놓기

» 사용량 제한 기능이 추가되어 기존 API와 호환되지 않음

- 이런 경우 새로운 버전의 라우터를 내놓으면 됨
- v2 라우터 작성(apiLimiter 추가됨)
- v1 라우터는 deprecated 처리(router.use로 한 번에 모든 라우터에 적용)

nodebird-api/routes/v1.js

```
const express = require('express');
const jwt = require('jsonwebtoken');

const { verifyToken, deprecated } = require('./middlewares');
const { Domain, User, Post, Hashtag } = require('../models');

const router = express.Router();

router.use(deprecated);

router.post('/token', async (req, res) => {
```

nodebird-api/app.js

```
...
const v1 = require('./routes/v1');
const v2 = require('./routes/v2');
const authRouter = require('./routes/auth');
...
app.use('/v1', v1);
app.use('/v2', v2);
app.use('/auth', authRouter);
...
```



4. 새 라우터 실행해보기

» nodecat의 버전 v2로 바꾸기

nodecat/routes/index.js

```
const express = require('express');  
const axios = require('axios');  
  
const router = express.Router();  
const URL = 'http://localhost:8002/v2';  
...
```

» v1 API를 사용하거나 사용량을 초과하면 에러 발생

▼ 그림 10-11 deprecated된 API 사용 화면

```
{ "code": 410, "message": "새로운 버전이 나왔습니다. 새로운 버전을 사용하세요." }
```

▼ 그림 10-12 사용량 초과 화면

```
{ "code": 429, "message": "1분에 한 번만 요청할 수 있습니다." }
```

10.7 CORS 이해하기



1. nodecat 프론트 작성하기

» 프론트에서 서버의 API를 호출하면 어떻게 될까?

- routes/index.js와 views/main.html 작성

nodecat/routes/index.js

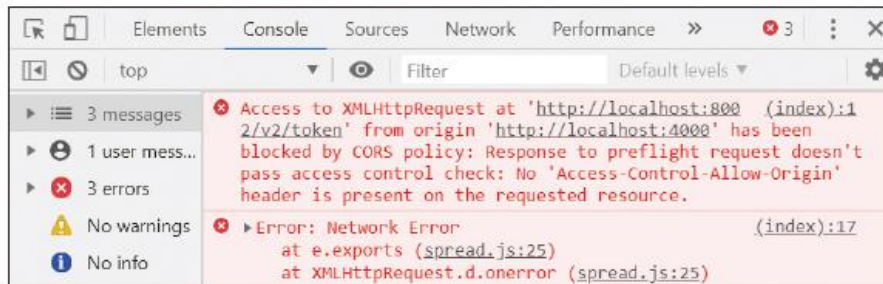
```
...
router.get('/', (req, res) => {
  res.render('main', { key: process.env.CLIENT_SECRET });
});

module.exports = router;
```

2. 프론트에서 요청 보내기

» localhost:4000에 접속하면 에러 발생

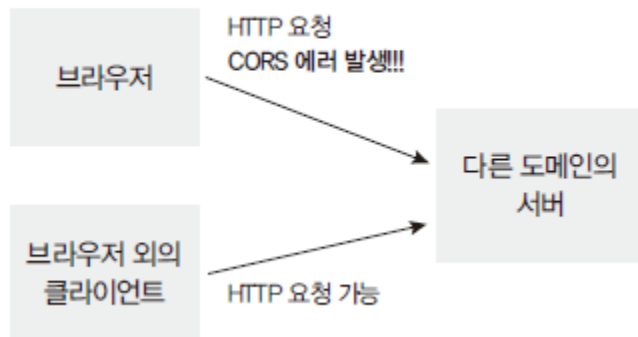
▼ 그림 10-13 Access-Control-Allow-Origin 에러



» 요청을 보내는 프론트(localhost:4000), 요청을 받는 서버(localhost:8002)가 다르면 에러 발생(서버에서 서버로 요청을 보낼때는 발생하지 않음)

- CORS: Cross-Origin Resource Sharing 문제
- POST 대신 OPTIONS 요청을 먼저 보내 서버가 도메인을 허용하는지 미리 체크

▼ 그림 10-14 CORS 에러



▼ 그림 10-15 Method가 OPTIONS

Name	Method	Status	Protocol	Type	Initiator	Size	Time	Waterfall
localhost	GET	304	http/1.1	do...	Other	156 B	11 ...	
axios.min.js	GET	302	h2	tex...	(index)	164 B	77 ...	
axios.min.js	GET	200	h2	scri...	axios.mi...	(dis...	12 ...	
token	OPTIONS	200	http/1.1	xhr	spread.js...	220 B	7 ms	

NodeBird API 콘솔

OPTIONS /v2/token 200 0.302 ms - 4



3. CORS 문제 해결 방법

» Access-Control-Allow-Origin 응답 헤더를 넣어주어야 CORS 문제 해결 가능

- res.set 메서드로 직접 넣어주어도 되지만 패키지를 사용하는게 편리
- npm i cors
- v2 라우터에 적용
- credentials: true를 해야 프론트와 백엔드 간에 쿠키가 공유됨

nodebird-api/routes/v2.js

```
const express = require('express');
const jwt = require('jsonwebtoken');
const cors = require('cors');

const { verifyToken, apiLimiter } = require('./middlewares');
const { Domain, User, Post, Hashtag } = require('../models');

const router = express.Router();

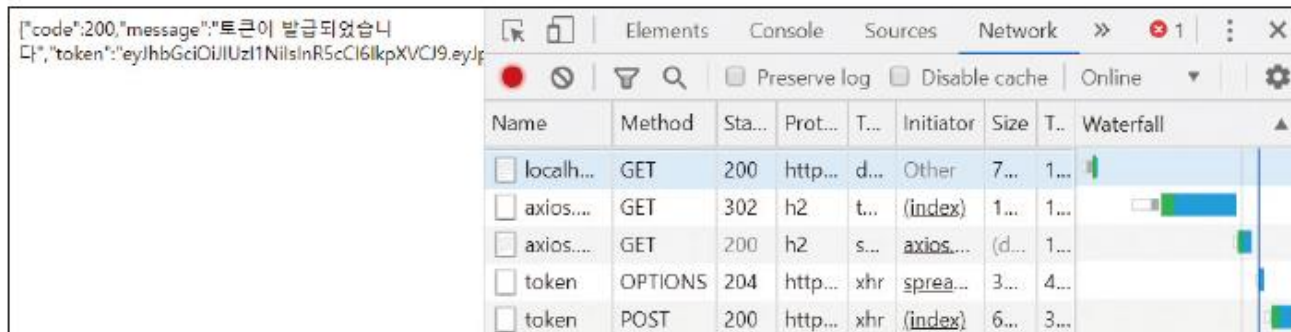
router.use(cors({
  credentials: true,
}));
...
```



4. CORS 적용 확인하기

» <http://localhost:4000>에 접속하면 정상적으로 토큰이 발급됨

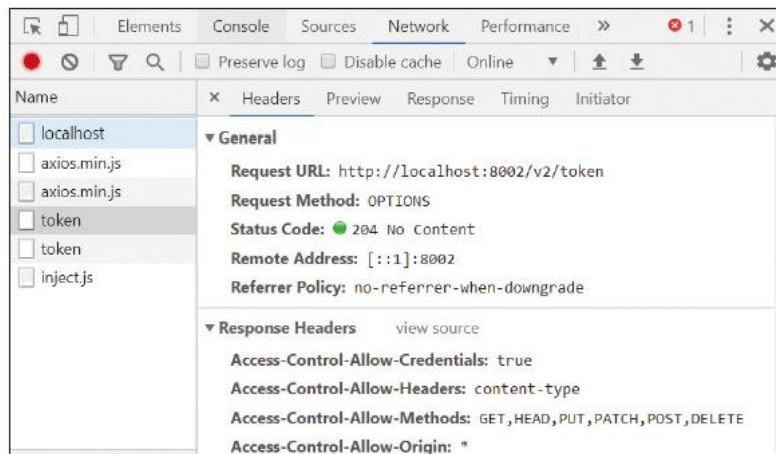
▼ 그림 10-16 CORS 요청 성공



» 응답 헤더를 보면 Access-Control-Allow-Origin 헤더가 들어 있음

- *은 모든 도메인을 허용함을 의미

▼ 그림 10-17 Access-Control-Allow-Origin: *





5. 클라이언트 도메인 검사하기

» 클라이언트 환경에서는 비밀키가 노출됨

- 도메인까지 같이 검사해야 요청 인증 가능
- 호스트와 비밀키가 모두 일치할 때만 CORS를 허용
- 클라이언트의 도메인(req.get('origin'))과 등록된 호스트가 일치하는 지 찾을 것
- url.parse().host는 http같은 프로토콜을 떼어내기 위함
- cors의 인자로 origin을 주면 * 대신 주어진 도메인만 허용할 수 있음

nodebird-api/routes/v2.js

```
const express = require('express');
const jwt = require('jsonwebtoken');
const cors = require('cors');
```

```
const url = require('url');
```

```
const { verifyToken, apiLimiter } = require('./middlewares');
const { Domain, User, Post, Hashtag } = require('../models');
```

```
const router = express.Router();
```

```
router.use(async (req, res, next) => {
  const domain = await Domain.findOne({
    where: { host: url.parse(req.get('origin')).host },
  });
  if (domain) {
    cors({
      origin: req.get('origin'),
      credentials: true,
    })(req, res, next);
  } else {
    next();
  }
});
```

```
router.post('/token', apiLimiter, async (req, res) => {
  ...
```



7. 유용한 미들웨어 패턴 알아보기

» 위의 미들웨어를 아래처럼 수정 가능

- 아래처럼 쓰면 미들웨어 위 아래로 임의의 코드를 추가할 수 있음
- 활용 가능

```
router.use(cors());
```

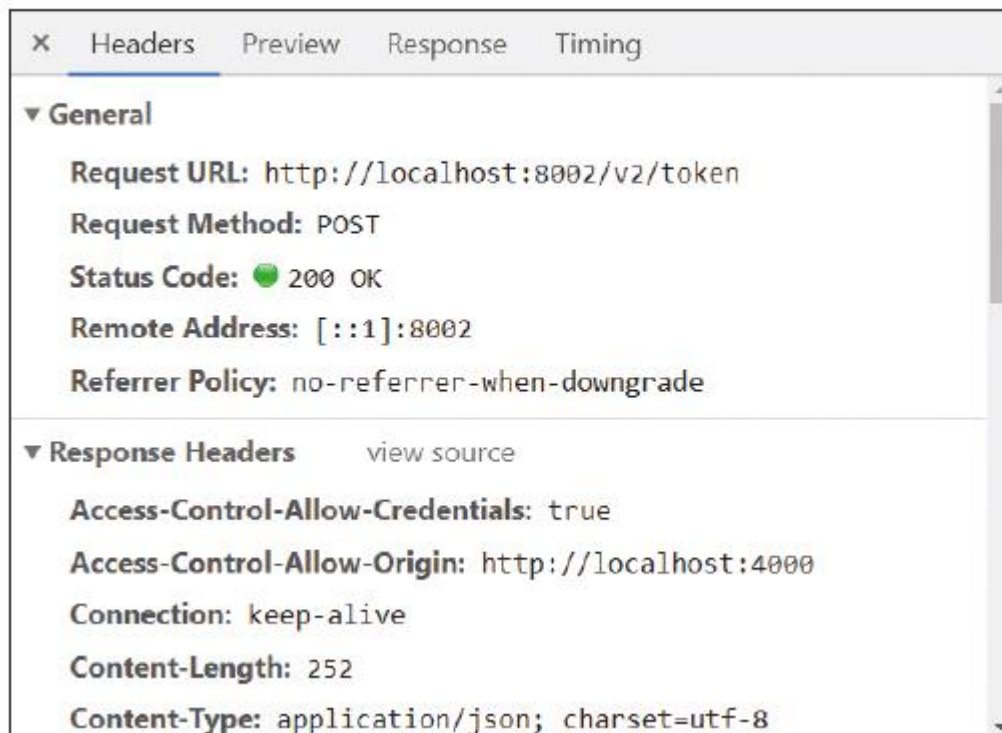
```
router.use((req, res, next) => {  
  cors()(req, res, next);  
});
```

8. CORS 요청 보내기

» localhost:4000에 접속

- 응답 헤더의 도메인 확인

▼ 그림 10-18 특정 도메인만 CORS 허용





9. 프록시 서버

» CORS 문제에 대한 또다른 해결책

- 서버-서버 간의 요청/응답에는 CORS 문제가 발생하지 않는 것을 활용
- 직접 구현해도 되지만 http-proxy-middleware같은 패키지로 손쉽게 연동 가능

▼ 그림 10-20 프록시 서버의 원리

